

Documentation technique – Gestion des congés

13 août 2026

1 Présentation

Le projet est organisé autour de trois services : **database** pour PostgreSQL, **backend** pour l'API Spring Boot et **frontend** pour l'interface React servie par Nginx. Le **backend** utilise le port 8080 et le **frontend** est accessible sur le port 5173.

2 Prérequis

Pour l'installation portable, le seul prérequis technique est Docker avec Docker Compose, ainsi qu'un navigateur Web. Java, Maven, Node.js et PostgreSQL ne sont pas nécessaires sur la machine d'accueil.

3 Lancement avec Docker

Depuis la racine du projet, exécuter :

```
./install.sh
```

Le script construit les images et démarre les services en arrière-plan. Le lancement direct est également possible :

```
docker compose up --build -d
docker compose ps
```

L'application est ensuite accessible à l'adresse `http://localhost:5173`. L'état attendu est Up pour backend et frontend, et Up (healthy) pour database.

4 Arrêt et conservation des données

Pour arrêter l'application sans supprimer les données :

```
./stop.sh
```

Cette commande utilise `docker compose down` et conserve le volume `postgres_data`. La commande suivante est destructive pour les données de la base et ne doit être utilisée que pour une réinitialisation volontaire :

```
docker compose down -v
```

5 Données de démonstration

Le script `backend/demo-data.sql` contient des données de démonstration et des comptes dont le mot de passe local est `password`. Ce script est prévu pour être importé manuellement dans la base après la création des tables; il n'est pas exécuté automatiquement par `docker compose`.

Les accès principaux sont les suivants :

- **Agent** : `demo.leila@cni.tn / password`;
- **Responsable** : `demo.responsable3@cni.tn / password`;
- **Administrateur** : `admin@test.com / test`.

Les données de démonstration sont réservées au test local et ne doivent pas être utilisées en production. La collection de tests se trouve dans `docs/gestion-conges.postman_collection.json`.

6 Lancement manuel

Le lancement manuel est réservé au développement. Il ne faut pas exécuter simultanément le `backend` manuel et le conteneur `backend`, car ils utilisent tous les deux le port 8080.

Pour lancer le `backend` seul :

```
cd backend
./mvnw spring-boot:run
```

Pour lancer le `frontend` séparément :

```
cd frontend
npm install
npm run dev
```

7 Fonctionnalités par rôle

- **Agent** : tableau de bord, consultation du solde, création, suivi et annulation d'une demande autorisée.
- **Responsable** : consultation des demandes assignées, approbation, refus, commentaire et historique des validations.
- **Administrateur** : gestion des services, agents, responsables directs, statuts et ajustements de solde.

8 Dépannage

Pour consulter les journaux :

```
docker compose logs -f
```

Si le démarrage échoue avec un message indiquant que le port 8080 ou 5173 est déjà utilisé, il faut arrêter l'ancien serveur local ou modifier le port publié dans `docker-compose.yml`. Il ne faut pas supprimer le volume pour résoudre un conflit de port.

9 Sécurité et données

Les mots de passe de démonstration ne doivent pas être utilisés en production. La clé de signature JWT et les identifiants de base doivent être externalisés avant tout déploiement réel. Il ne faut pas exposer le secret JWT ni supprimer la base pour résoudre un problème de lancement.